

Moving the task from simulation to hardware

The main difference is that a command is no longer proof of movement. In ideal simulation, the arm can follow the requested angles exactly. A real motor may move slowly, stop short or fail. I would replace the simulation backend with a hardware driver that sends joint commands and reads encoder positions, speeds and faults. Before moving the arm, I would check motor direction, zero positions, gear ratios, units and joint limits against the model. The planner and GUI could keep the same basic interfaces.

Communication can become a bottleneck, especially when several motors share a bus. Reading and writing each motor separately may take too long for the chosen update rate. I would measure the actual communication time, use grouped reads and writes where supported, and keep device communication separate from planning and drawing the GUI. Every reading needs a timestamp so old data can be detected. A watchdog on the device or low-level controller should stop or safely hold the arm when commands stop arriving. A crashed ROS program cannot reliably send its own stop command. Tight timing requirements may need a real-time control layer rather than a Python timer.

The control loop also needs testing with real loads. Gravity, friction, backlash and the object's weight can change how the arm responds. I would begin with slow movements and conservative speed, acceleration and current limits, then tune the controller using measured error. The integral term should be bounded, and motor limits must be respected. A smooth reference does not guarantee smooth physical motion. As in this project, reaching a target should depend on measured position and speed settling within limits, with a timeout if that does not happen.

Safety needs more than the planar ground check. Real links and the gripper have thickness, measurements have errors, and the work area may contain obstacles or people. I would add suitable clearance margins and collision checks, define a controlled homing procedure, and provide an independently effective emergency stop. The GUI cancel button is not a replacement for an emergency stop. Simulation reset must never become a command that instantly jumps a physical arm to its starting angles.

Finally, the gripper needs feedback. The current software grasp is ideal, and Gazebo uses an attachment constraint rather than a friction-based grasp. Hardware needs a way to confirm that the object was picked up and released, and a recovery plan for missed grasps or dropped objects. The GUI should clearly show requested positions, measured positions, stale data and faults. I would test in stages: driver checks, unloaded low-speed moves, known targets, and then controlled pick-and-place trials. I would review the recorded results at each stage before claiming that the simulated task works on hardware.